

Comprehensive Internship Project Report: ClipMind AI

Advanced AI-Powered Video Summarization & Analytics Platform

Prepared By: Devanshi Malhotra
Role: AI & Full-Stack Engineering Intern
Date: August 2026
Company: Infosys (Springboard Internship)

1. Executive Summary

In the modern digital era, the proliferation of video content across educational, corporate, and social domains has led to significant challenges in information retrieval. Users frequently spend hours scrubbing through lengthy videos to extract critical insights. **ClipMind AI** was conceptualized and developed to bridge this gap.

ClipMind AI is an intelligent, scalable, and fully containerized web application that automates the extraction of insights from long-form video content. By leveraging state-of-the-art Large Language Models (LLMs) and advanced Speech-to-Text inference engines via the Groq LPU API, the platform dynamically generates highly accurate transcripts, tiered summaries, and interactive timestamped key moments.

This report documents the end-to-end software development lifecycle of the ClipMind AI platform, detailing the architectural decisions, database modeling, AI integration strategies, frontend UI/UX design, and final cloud deployment.

2. Introduction & Problem Statement

2.1 Background

The widespread adoption of online learning and remote work has resulted in thousands of hours of video being generated daily. However, unlike text documents, video is inherently difficult to skim or search.

2.2 The Problem

- Time Inefficiency:** Students and professionals waste time watching irrelevant portions of videos to find specific answers.
- Accessibility Barriers:** Lack of accurate transcription hinders accessibility for hearing-impaired users.
- Content Management:** Educators lack automated tools to convert their video lectures into readable study guides.

2.3 Objectives

- Build a full-stack web application capable of ingesting local videos and YouTube links.
- Implement a robust RBAC (Role-Based Access Control) system tailored for Administrators, Educators, Learners, and Content Creators.
- Develop a multi-stage AI pipeline (Audio Extraction -> Transcription -> Summarization).
- Ensure the application is cloud-native and highly scalable.

Landing Page Hero

Figure 1: ClipMind AI Landing Page Interface

3. System Architecture & Tech Stack

ClipMind AI adopts a decoupled, microservices-oriented monolithic architecture. The frontend handles UI rendering and state management, while the backend is strictly an API provider handling heavy AI processing.

System Architecture

Figure 2: Comprehensive Architecture Diagram

3.1 Frontend (Client-Side)

- Next.js (React 18):** Chosen for its hybrid SSR/CSR rendering capabilities, providing excellent SEO and rapid page loads.
- Tailwind CSS:** Used for utility-first responsive styling, featuring a custom dark-mode UI with gradient accents.
- Lucide React:** Iconography.
- Recharts:** Used on the Administrator dashboard to visualize analytics.

3.2 Backend (Server-Side)

- FastAPI (Python 3.11):** Selected for its high performance (powered by Starlette and Pydantic) and native support for asynchronous programming, which is crucial for handling long-running AI tasks.
- FFmpeg & yt-dlp:** Utilized internally for robust multimedia processing and audio extraction.
- JWT (JSON Web Tokens):** For stateless, secure API authentication.

3.3 Database Layer (Polyglot Persistence)

The application utilizes a dual-database design to optimize for differing data constraints.

- PostgreSQL:** Handles strongly typed, relational data (Users, Roles, Video Metadata, Bookmarks).
- MongoDB Atlas:** Handles highly unstructured, large-payload document data (AI generated Transcripts, Summaries, JSON arrays of Key Moments).

PostgreSQL Admin View

Figure 3: Relational Database structure in pgAdmin

4. Database Schema & Modeling

4.1 Relational Schema (PostgreSQL)

Users Table

- `id` (Primary Key, Integer)
- `email` (String, Unique)
- `hashed_password` (String)
- `role` (Enum: `educator`, `learner`, `administrator`, `content_creator`)

Videos Table

- `id` (Primary Key, Integer)
- `owner_id` (Foreign Key -> `Users.id`)
- `title` (String)
- `filename` (String)
- `status` (Enum: `processing`, `completed`, `error`)
- `created_at` (DateTime)

4.2 Document Schema (MongoDB)

Since AI transcripts can reach hundreds of kilobytes, storing them in PostgreSQL would degrade performance. MongoDB is used as a document store.

MongoDB Storage

Figure 4: MongoDB Atlas storing complex JSON AI insights

Transcripts Collection

```
{
  "_id": "60d5ec49f1",
  "video_id": 5,
  "transcript": [
    { "start": 0.0, "end": 4.5, "text": "The sun shines during the day." },
    { "start": 4.5, "end": 8.0, "text": "Miss Matthews will argue the affirmative." }
  ]
}
```

5. Artificial Intelligence Integration

The core value proposition of ClipMind AI relies on its multi-stage AI pipeline.

5.1 Step 1: Audio Extraction

When a video is uploaded, the FastAPI backend delegates the file to `ffmpeg-python`. The video is stripped of its visual track, and the audio is downsampled to a 16kHz Mono MP3 file. This drastically reduces the payload size before it is sent to the LLM APIs.

5.2 Step 2: High-Speed Transcription (Whisper-Large-V3)

The extracted MP3 is sent to the **Groq Cloud API** running the `whisper-large-v3` model. Groq's Language Processing Units (LPUs) are utilized to achieve inference speeds vastly superior to traditional GPUs.

API Integration Snippet:

```
def _groq_transcribe(audio_path: str):
    client = get_groq_client()
    with open(audio_path, "rb") as file:
        transcription = client.audio.transcriptions.create(
            file=(audio_path, file.read()),
            model="whisper-large-v3",
            response_format="verbose_json",
        )
    return transcription
```

5.3 Step 3: Deep Summarization (LLaMA-3)

Once the transcript is generated, the raw text is concatenated and injected into a prompt engineered for `llama3-70b-8192`. The prompt enforces strict JSON output, ensuring the LLM returns a structured schema containing a `short_summary`, `detailed_summary`, and an array of `key_moments` with precise timestamps.

AI Processing UI

Figure 5: Frontend UI displaying real-time processing status

6. Detailed Feature Breakdown

6.1 Administrator Controls & Analytics

Administrators have access to a global view of the platform. Using Recharts, the dashboard renders visualizations of system usage, trending keywords across all processed videos, and user registration velocity.

Admin Dashboard

Figure 6: Global Analytics Dashboard for Administrators

6.2 Interactive Video Dashboard

The primary user interface consists of a dynamic video player synced with the AI outputs.

- **Smart Transcripts:** A searchable transcript window. Clicking on any transcript line seeks the video to that exact millisecond.
- **Key Moments:** AI-identified chapters are displayed as clickable buttons below the video.

Transcript UI

Figure 7: Interactive Transcript synced with the video player

Summary UI

Figure 8: AI-generated Summary tab

6.3 Content Management & Uploading

Educators and Content Creators can upload media via two methods:

1. **Direct File Upload:** .mp4, .mov, .avi.
2. **YouTube Import:** By providing a URL, the backend utilizes `yt-dlp` to securely stream and download the video directly to the server, bypassing client-side bandwidth limitations.

Upload Interface

Figure 9: Upload interface supporting local files and YouTube URLs

7. Performance Evaluation & Benchmarking

To ensure the platform could scale, extensive benchmarking was conducted using Jupyter Notebooks (`testing/evaluate_pipeline.ipynb`).

The evaluation tested the pipeline against videos of varying lengths (1 minute to 15 minutes).

Results:

- **Local Inference (RTX 3060):** A 5-minute video required ~45 seconds for transcription and ~30 seconds for LLaMA summarization.
- **Groq LPU Inference:** A 5-minute video required **<3 seconds** for transcription and **<2 seconds** for LLaMA summarization.

By decoupling the AI inference and delegating it to Groq, the backend server CPU overhead was reduced by 95%, allowing a single basic server instance to handle dozens of concurrent video uploads.

Evaluation Script

Figure 10: Jupyter Notebook evaluating latency and accuracy

8. Deployment & CI/CD Strategy

ClipMind AI is fully containerized and deployed on the Render cloud platform.

8.1 Dockerization

Both the frontend and backend possess highly optimized Dockerfiles.

- **Frontend Dockerfile:** Utilizes multi-stage builds. It builds the static Next.js assets in a Node environment and copies only the standalone execution files to a lightweight Alpine image.
- **Backend Dockerfile:** Installs system-level dependencies (`ffmpeg`) alongside Python dependencies via `pip`.

8.2 Live Environments

- **Frontend URL:** <https://clipmind-ai-frontend.onrender.com/> (<https://clipmind-ai-frontend.onrender.com/>)
- **Backend API URL:** <https://clipmind-ai-8hxx.onrender.com/> (<https://clipmind-ai-8hxx.onrender.com/>)

8.3 Technical Constraints Addressed

During cloud deployment on Render's free tier, several architectural challenges were successfully navigated:

1. **Ephemeral Storage:** Handled by designing the database to hold all critical data, ensuring that if the container restarts and wipes the local `/uploads` directory, the AI insights survive.
2. **Cold Starts:** Managed by implementing frontend loading states and retry logic.

9. Conclusion

The ClipMind AI project successfully synthesized modern web development practices with cutting-edge Generative AI APIs. It resulted in a highly functional, beautifully designed, and deeply technical platform capable of drastically reducing the time required to digest video content.

Throughout the internship, significant skills were acquired across the full stack—from engineering complex SQL/NoSQL architectures to fine-tuning LLM prompts and configuring Docker deployment pipelines. The platform stands as a robust minimum viable product (MVP) ready for future real-world scaling.

10. Future Enhancements

While the core objectives have been achieved, the platform is designed with extensibility in mind. Future iterations will aim to implement:

1. **Retrieval-Augmented Generation (RAG):** Allowing users to open a chatbox and "talk" directly to the video, asking hyper-specific questions (e.g., "At what time did the speaker mention the Q3 revenue?").
2. **AWS S3 Integration:** Offloading physical video storage from the application server to a dedicated S3 bucket to bypass ephemeral storage limitations permanently.
3. **Multi-Language Support:** Utilizing Whisper's translation capabilities to automatically dub or subtitle videos into secondary languages.

End of Document